Advanced Hardware Engineering

Lab 02: MicroBlaze IP

Build your own microcontroller on an FPGA and run your first embedded application

Prof. Dr.-Ing. Ali Hayek

29.10.2026

Goals and learning objectives

Goal

Build your own microcontroller unit on the FPGA and run a first C program on it.

Skills

IP Integrator, Block and Connection Automation, UART and interrupts, HDL wrapper, hardware export, Vitis platform and application.

Deliverables

Screenshot of your block design, utilization and timing summary, and the serial terminal output of your program.

Background: the MicroBlaze processor

- 32 general-purpose registers, each 32 bits wide
- 32-bit instruction word with three operands and two addressing modes
- 32-bit address bus
- Single-issue pipeline with 3 or 5 stages (configurable)

32 Bit microcontroller on FPGA

A soft processor is described in HDL and built from FPGA logic. The Artix-7 has no hard CPU, so we build one. Size and features can be configured.

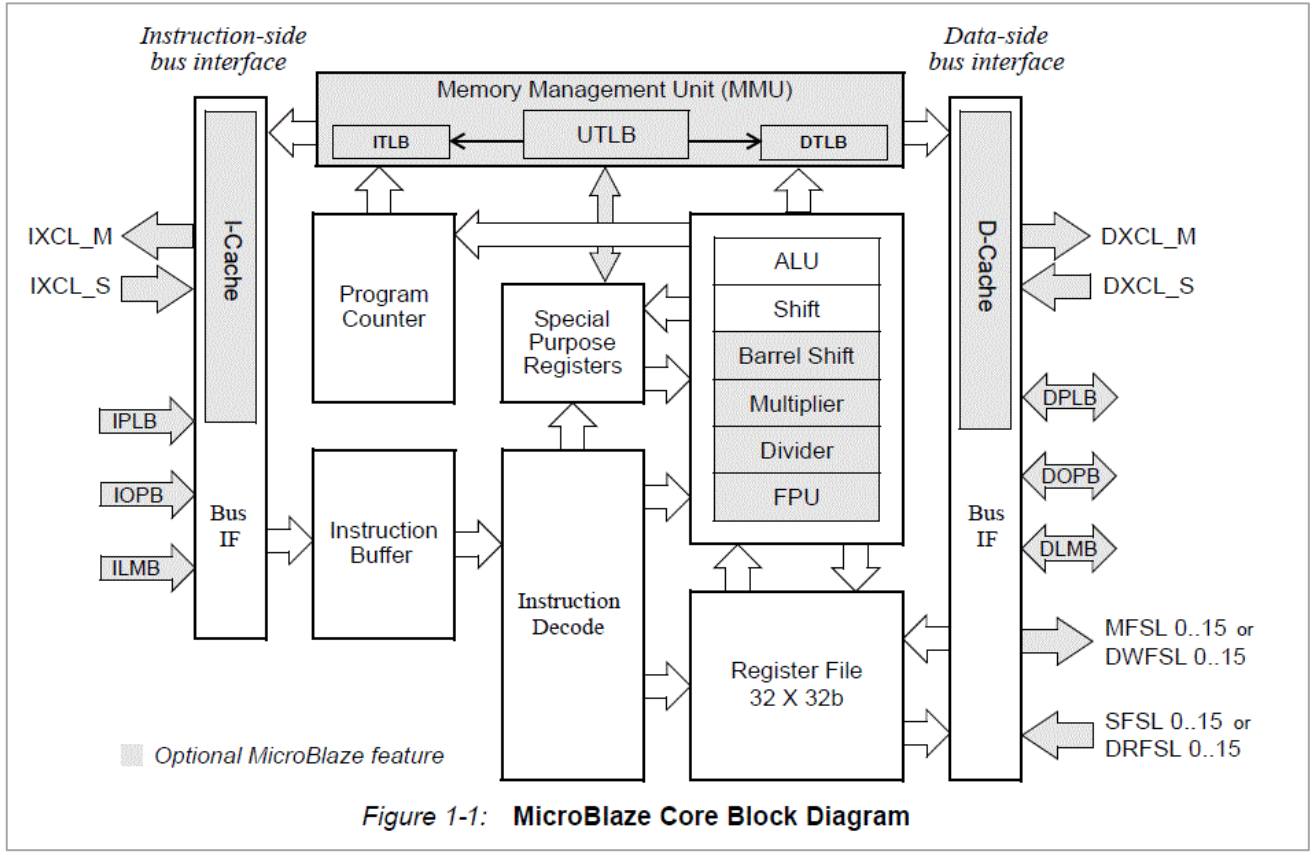
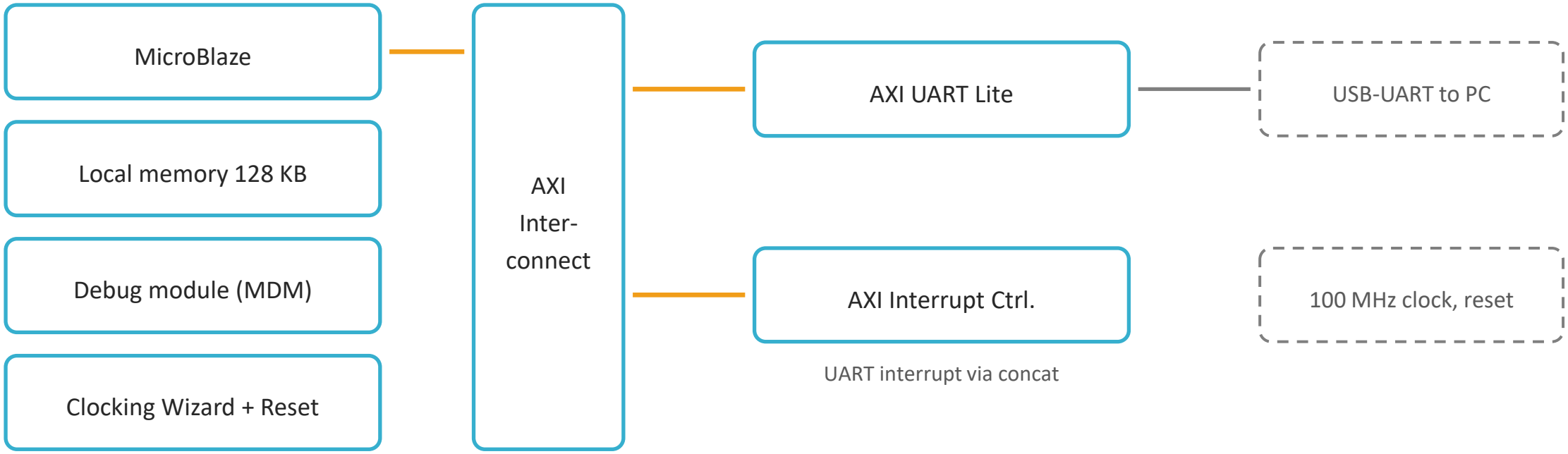


Figure: MicroBlaze core block diagram

What we will build



Catalog IP only: you assemble the system, you do not write HDL.

The building blocks explained

Block	Role	Why it is needed
MicroBlaze	Soft CPU that runs your C program	The "brain" of the system
Local memory (LMB)	Block RAM for program and data	The program lives inside the FPGA
Debug module (MDM)	JTAG access for download and debug	Lets Vitis load and debug software
AXI Interconnect	Connects the CPU to peripherals	The "bus" with addresses
AXI UART Lite	Serial port to the PC	Text input and output
AXI Interrupt Controller	Collects interrupts for the CPU	Peripherals can signal events
Clocking Wizard, Reset	Clock and reset for all blocks	Everything needs a clean clock

Tools and preparation

Item	Details
Tools	Vivado and Vitis of the same release (this tutorial was prepared with an older version, should work with 2026)
Board files	Digilent board files for Vivado, so the Nexys A7 can be selected as a board
Hardware	Nexys A7 board and micro USB cable (power, JTAG and UART)
Serial terminal	Terminal view in Vitis, or PuTTY; 8 data bits, no parity, 1 stop bit
Reference	MicroBlaze Processor Reference Guide (search "UG081" on docs.amd.com)

Digilent Nexys A7 page: digilent.com/reference/programmable-logic/nexys-a7/start

Digilent IP Integrator guide: digilent.com/reference/programmable-logic/guides/getting-started-with-ipi

Note: the screenshots come from an earlier tool release; menus in your release can look different.

Task 1: build the hardware

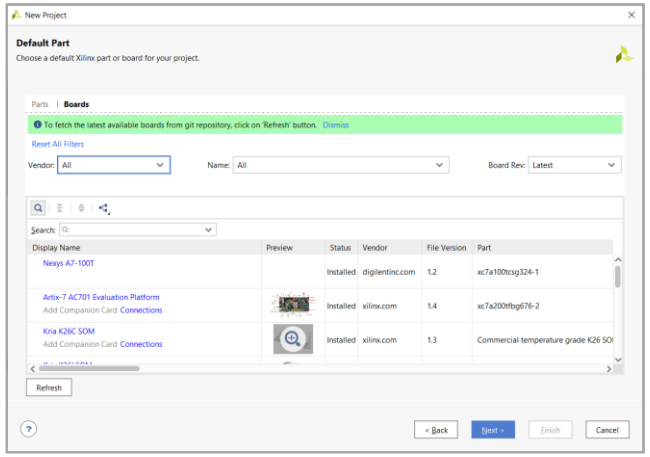
1. **Create the project and block design** – select the board, start IP Integrator
2. **Add and configure the MicroBlaze** – Block Automation, clock settings
3. **Connect the design** – Connection Automation, UART, interrupt
4. **Validate and create the HDL wrapper** – check the design, create the top module
5. **Generate the bitstream and export the hardware** – XSA file for Vitis

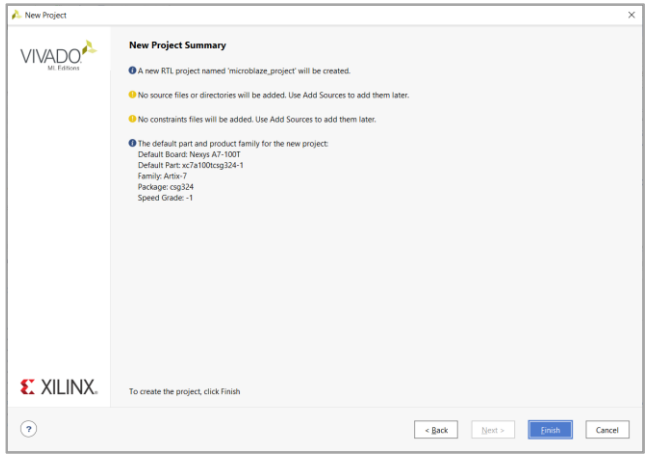
Hint

In Task 1 you decide what hardware exists inside the FPGA. The generated XSA file (Xilinx Support Archive) is a file exported from Vivado that describes your FPGA hardware system.

1.1 Create the Vivado project

1. Start Vivado and select Create Project
2. Give the project a name and a short path; select RTL Project
3. On the board page select the Nexys A7 (after installing the board files)
4. Check the summary and click Finish
5. Then choose Create Block Design (IP Integrator)



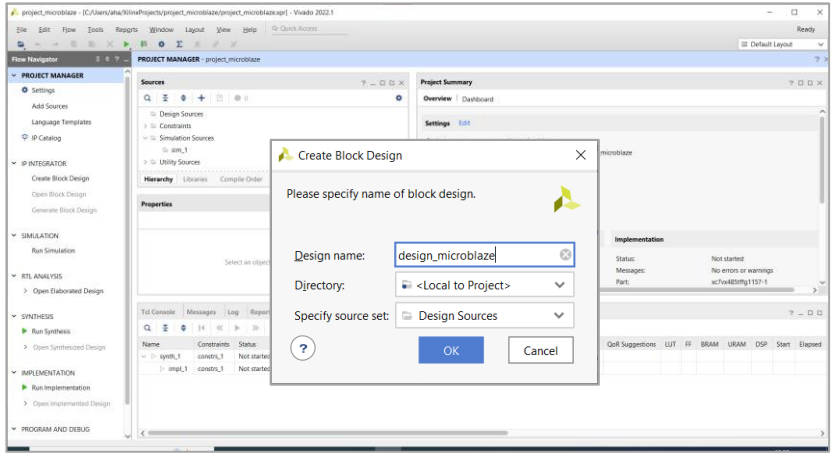


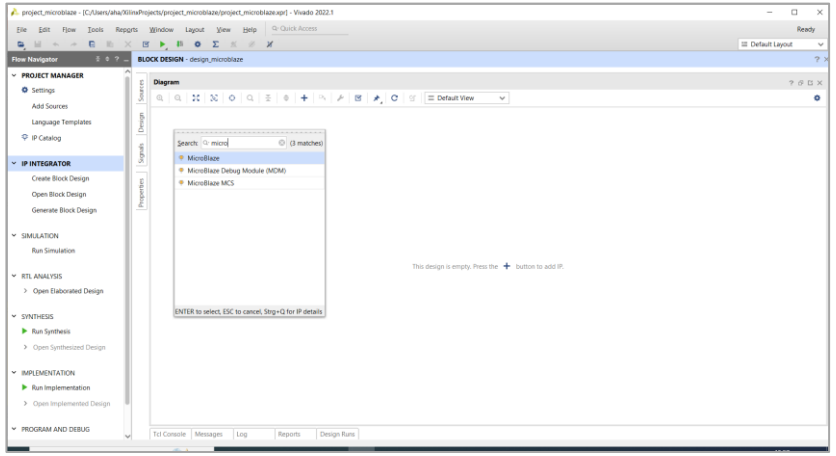
Hint

A project collects sources and settings. Selecting the board, not just the FPGA part, gives Vivado the pin and clock information of the Nexys A7.

1.2 Create the block design and add MicroBlaze

1. Create Block Design: enter a name, e.g. design_microblaze
2. Press the + button (or Add IP) in the diagram
3. Search for MicroBlaze and double-click it
4. The MicroBlaze block appears in the diagram



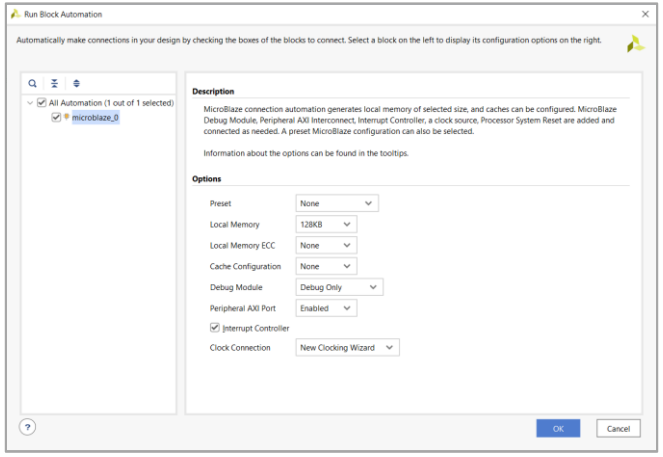


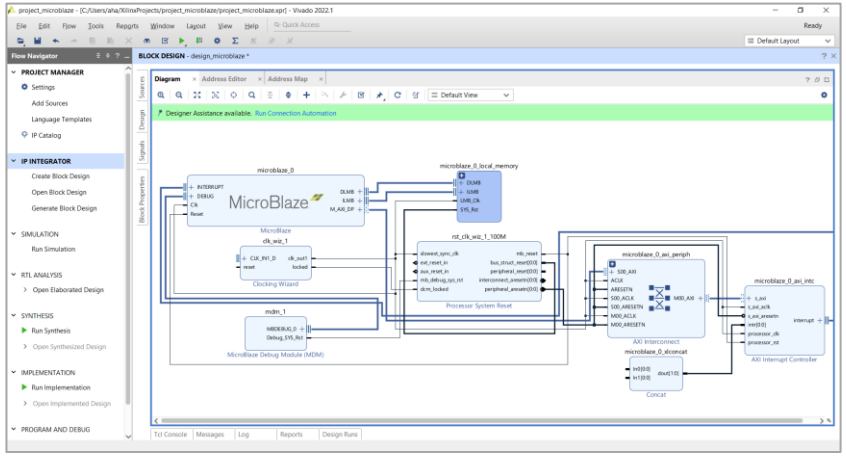
Hint

IP Integrator is the graphical tool to assemble a system from IP blocks. Each block is an IP core; you connect them like components on a board.

1.3 Configure the MicroBlaze with Block Automation

1. Click Run Block Automation in the green banner
2. Select Local Memory 128 KB
3. Activate the interrupt controller
4. Keep the other options, click OK
5. Check the automatically generated blocks



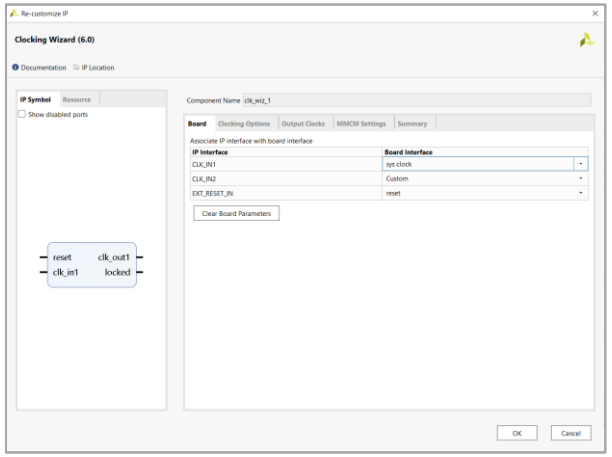


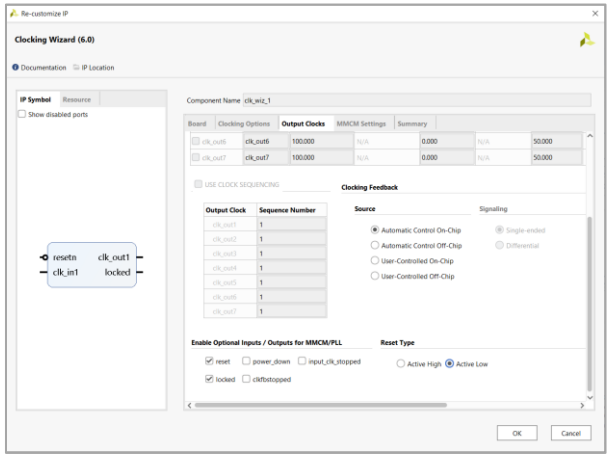
Hint

Block Automation adds everything a processor needs to run: local memory, debug module, clock and reset logic, and the interrupt controller you selected.

1.4 Adjust the clock settings

1. Double-click the clock module (Clocking Wizard) in the diagram
2. On the Board tab check that the input clock is the board clock
3. On the Output Clocks tab check the output frequency and the reset type
4. Click OK



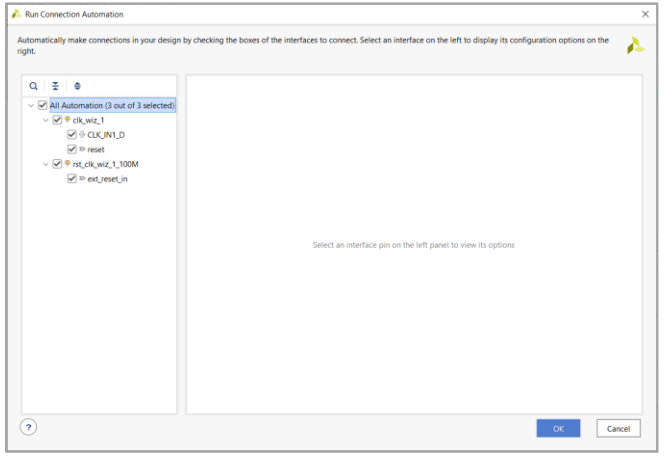


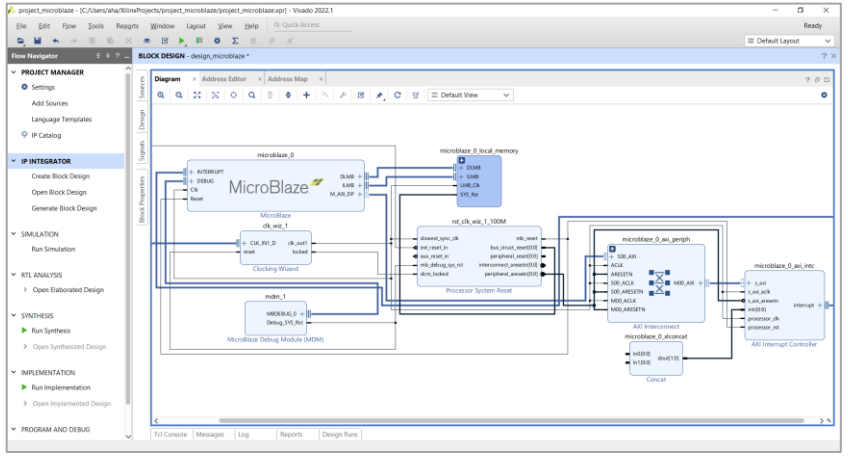
Hint

All blocks run on this clock. The Clocking Wizard turns the board clock into the system clock and defines how the reset is generated.

1.5 Connect the design with Connection Automation

1. Click Run Connection Automation
2. Select all signals in the list
3. Click OK
4. Check the diagram: almost all signals are now connected



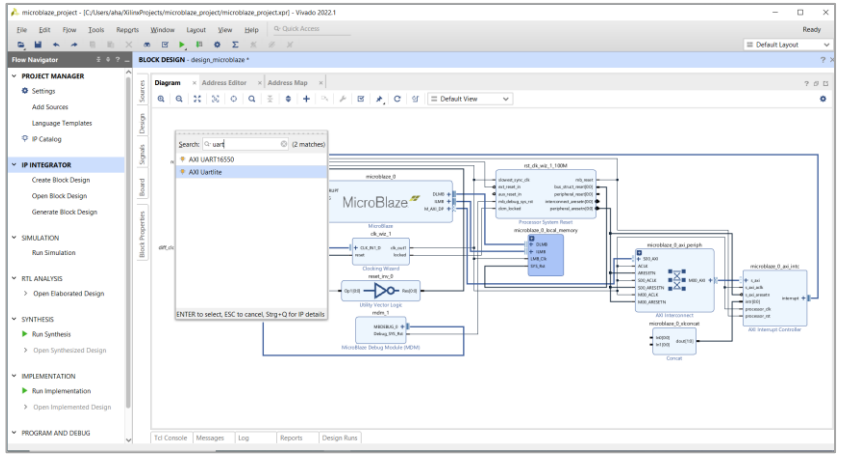


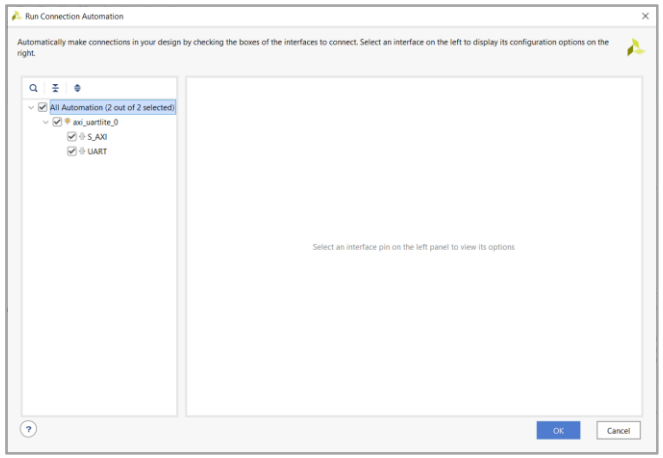
Hint

Connection Automation wires clocks, resets and AXI connections automatically and connects board interfaces to external ports.

1.6 Add the UART

1. Press + and search for UART
2. Add AXI UART Lite
3. Run Connection Automation again
4. Select the UART interface and click OK
5. The UART is now connected



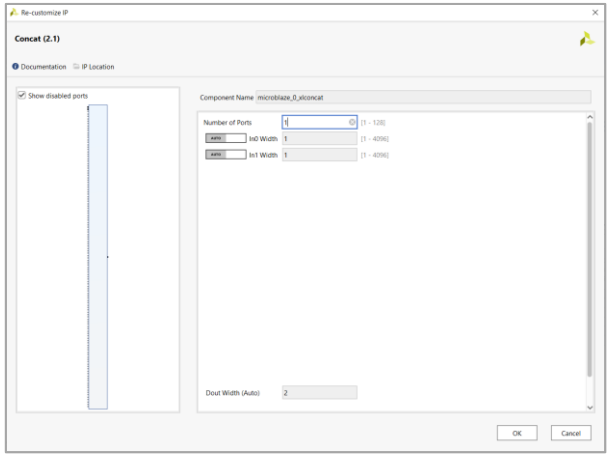


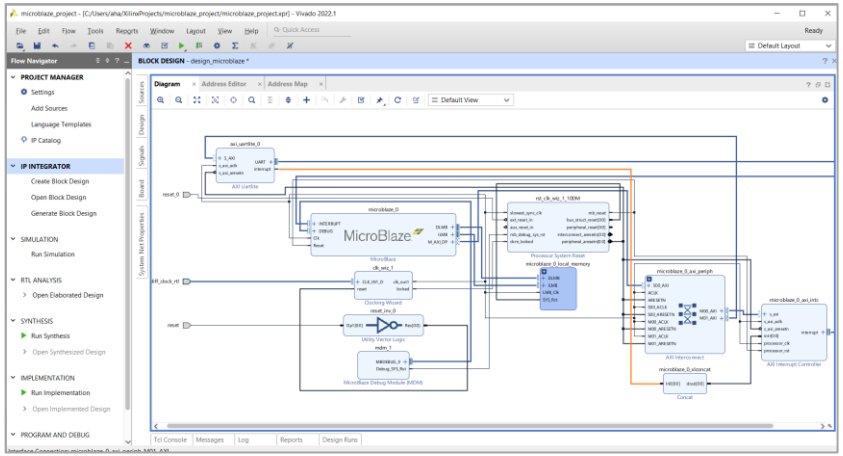
Hint

A UART is a serial port. It lets the program print text to your PC through the USB cable of the board. Without it you cannot see any output.

1.7 Connect the interrupt

1. The UART interrupt pin has to be connected manually
2. Double-click the concat block of the interrupt controller
3. Set the number of ports to 1
4. Connect the UART interrupt output to the concat input



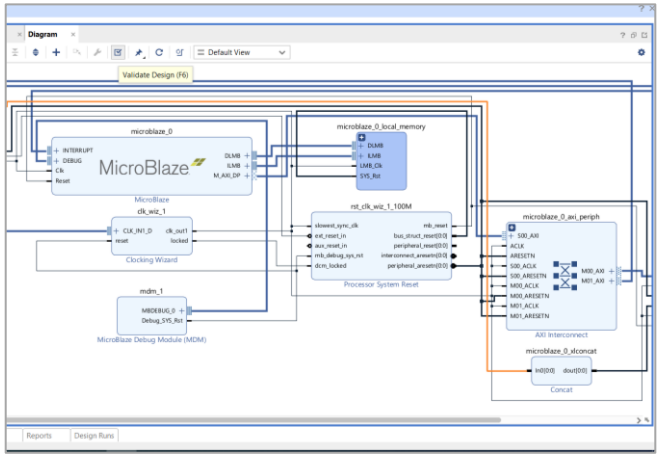


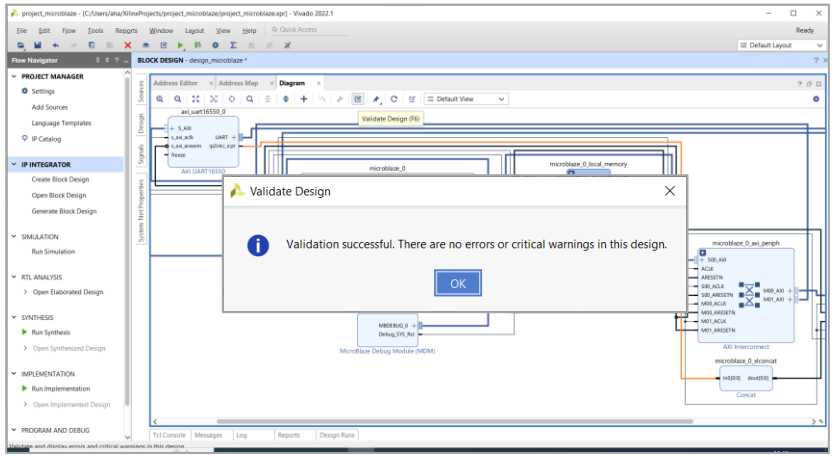
Hint

An interrupt lets a peripheral tell the CPU that something happened. The concat block gathers all interrupt lines into the input of the interrupt controller.

1.8 Validate the design

1. Click the Validate Design button in the toolbar (or press F6)
2. Read the message: validation should be successful
3. If there are errors, read the messages and run Connection Automation again



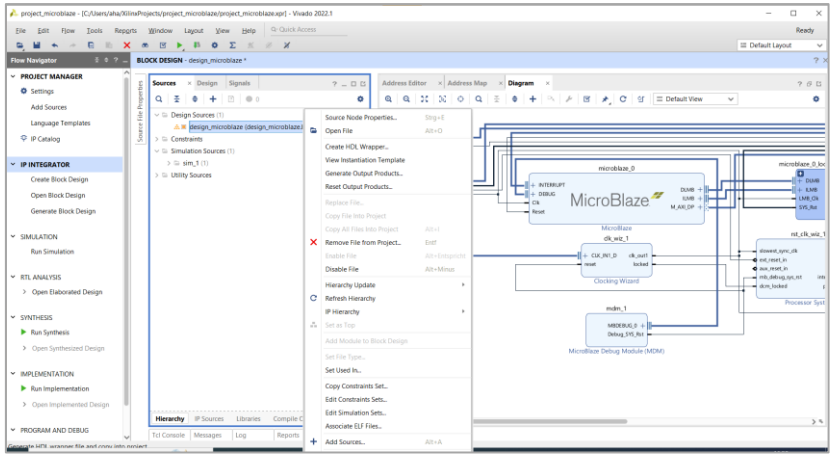


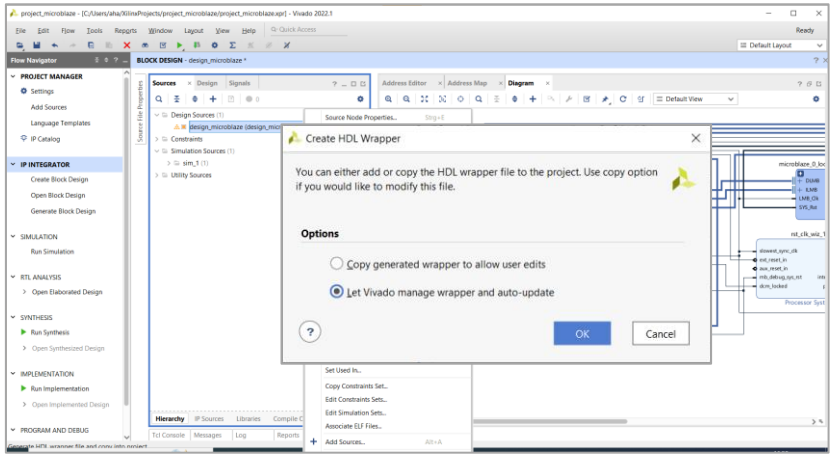
Hint

Validation checks that all connections, clocks, resets and addresses are consistent. It is much faster than finding the error after synthesis.

1.9 Create the HDL wrapper

1. In the Sources window right-click the block design
2. Select Create HDL Wrapper
3. Choose Let Vivado manage wrapper and auto-update, click OK
4. Open the wrapper file to inspect it



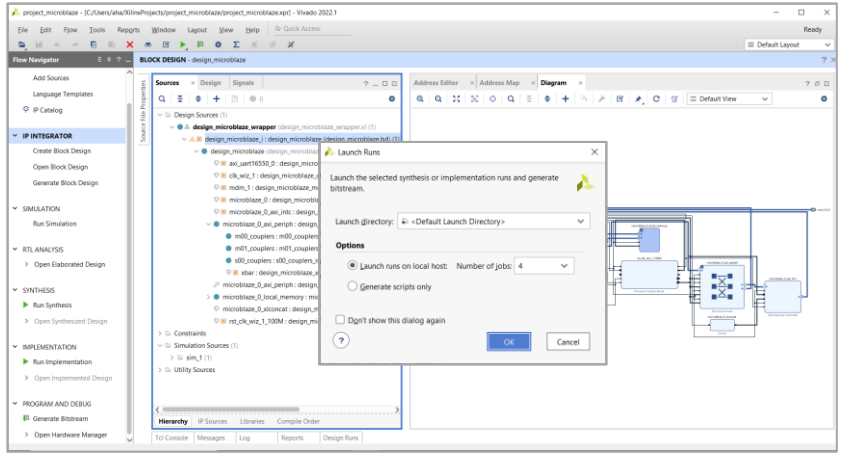


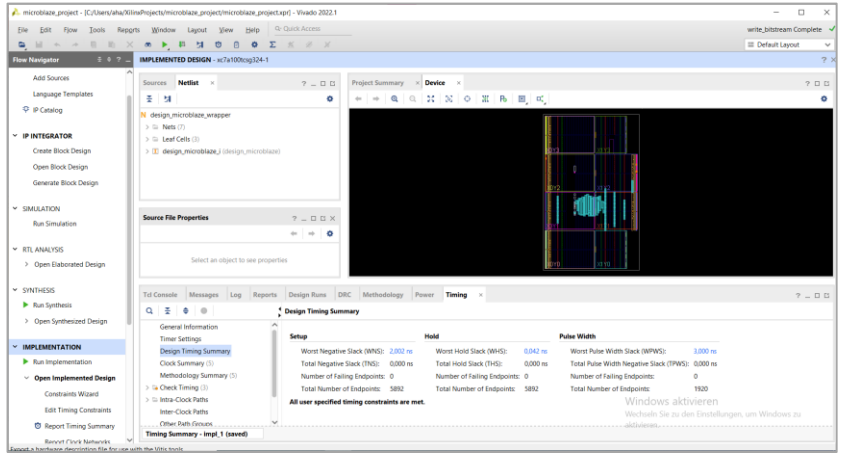
Hint

The block design is graphical. Synthesis needs HDL, so Vivado generates a top-level HDL file (the wrapper) that instantiates the whole block design.

1.10 Synthesis, implementation and bitstream

1. Click Generate Bitstream in the Flow Navigator
2. Accept the dialog that runs synthesis and implementation
3. Wait until the process is finished (a few minutes)
4. Open the timing summary to check that timing is met



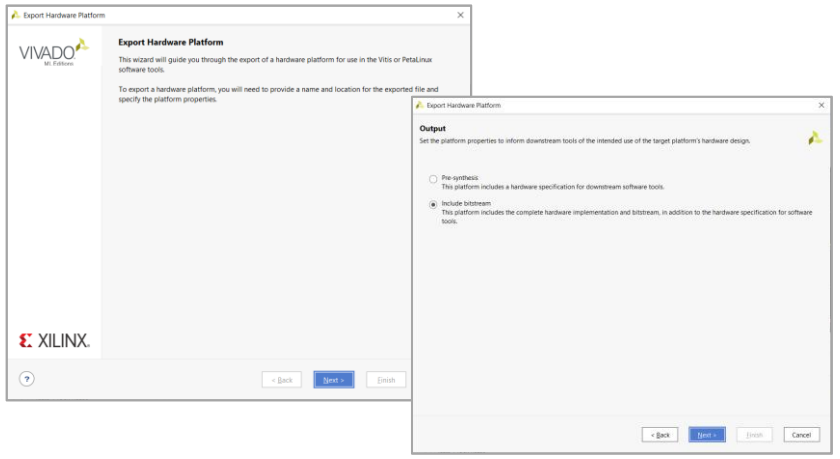


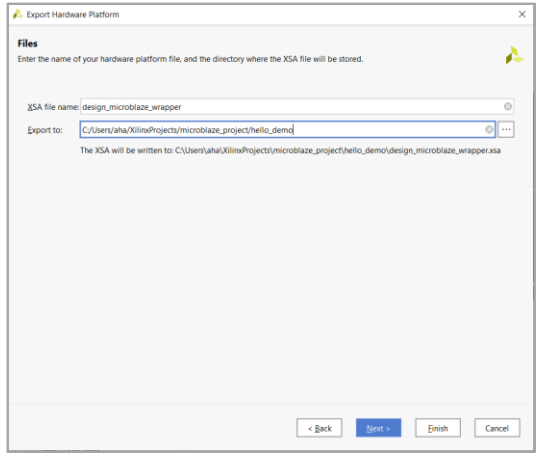
Hint

Synthesis turns the design into logic elements, implementation places and routes them on the chip, and the bitstream is the file that configures the FPGA.

1.11 Export the hardware to Vitis

1. Select File, Export, Export Hardware
2. Choose a fixed platform and include the bitstream
3. Select a name and a location for the XSA file
4. Click Finish





Hint

The XSA file is the handoff from hardware to software. It contains the hardware description, the address map of all peripherals and the bitstream.

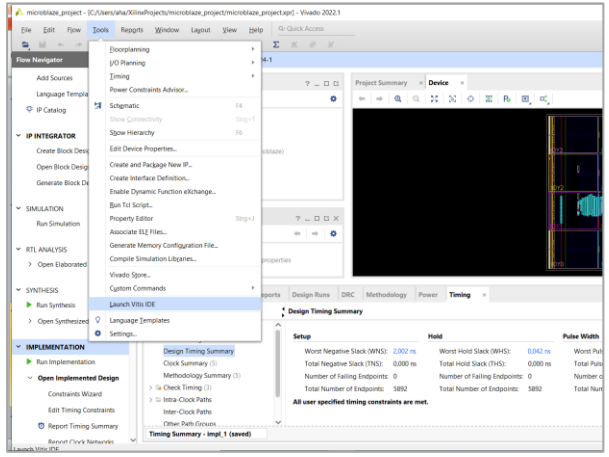
Task 2: write the first embedded application

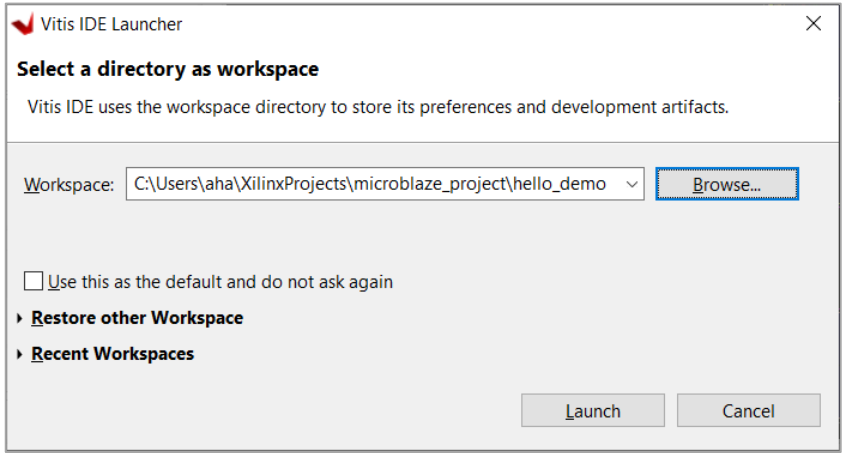
1. **Launch Vitis and select a workspace** – a folder for your software projects
2. **Create and build the platform project** – from the XSA file of Task 1
3. **Create the application project** – Hello World template, then build
4. **Program the FPGA** – download the bitstream
5. **Run the program and read the terminal** – your message appears

Hint
The platform describes the hardware to the software tools. The application is the C program that runs on the MicroBlaze.

2.1 Launch Vitis and select a workspace

1. In Vivado select Tools, Launch Vitis IDE
2. Choose a workspace folder with a short path
3. Click Launch
4. If Vitis is missing, add it to your installation



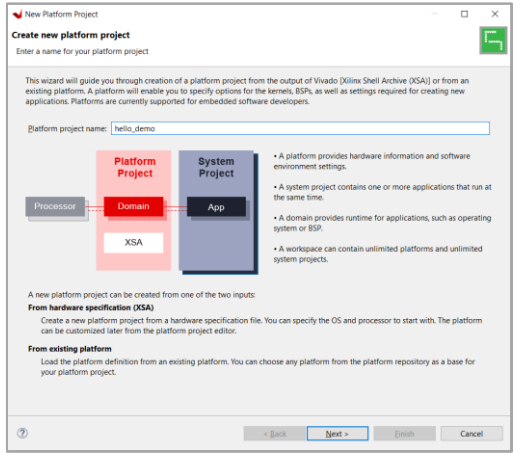


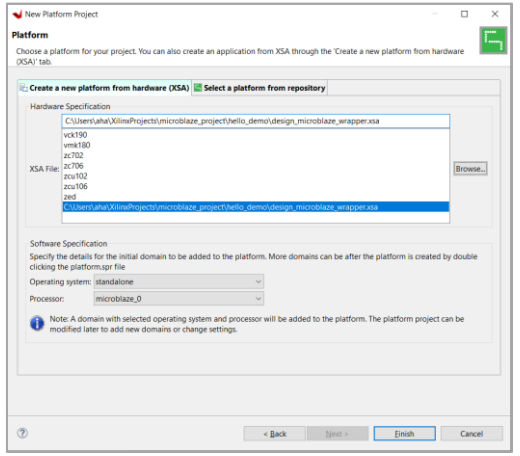
Hint

A workspace is the folder where Vitis stores your platform and application projects.

2.2 Create the platform project

1. In the welcome page choose Create Platform Project
2. Enter a platform name, e.g. hello_demo
3. Select Create from hardware specification (XSA) and browse to your XSA file
4. Choose the standalone operating system and the processor microblaze_0
5. Click Finish



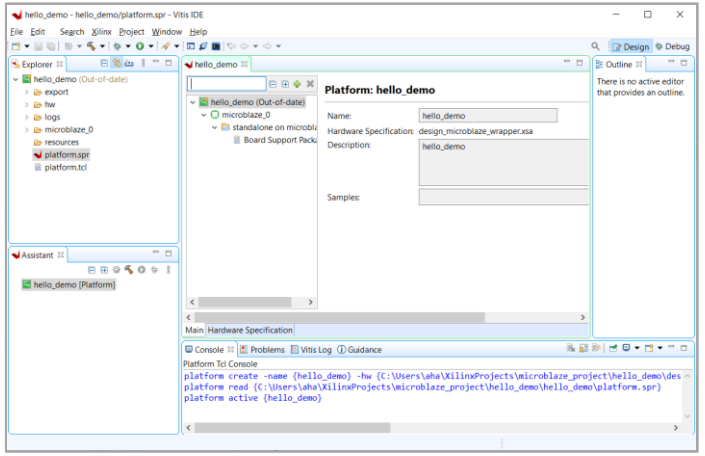


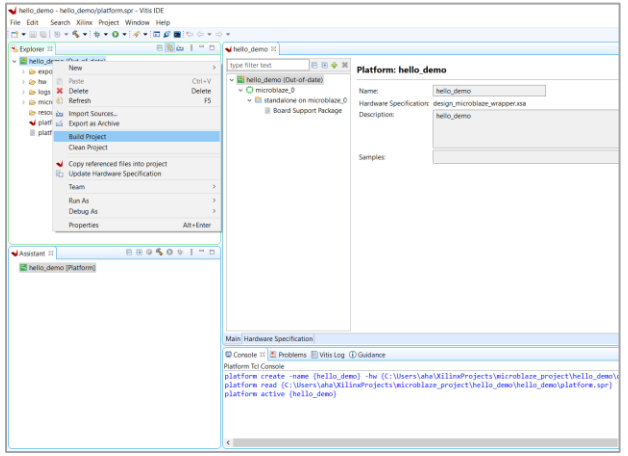
Hint

The platform project is the hardware description plus the board support package (BSP): drivers and a small standard library. The application is built on top of it.

2.3 Build the platform

1. Right-click the platform project
2. Select Build Project
3. Wait for the console to report a successful build



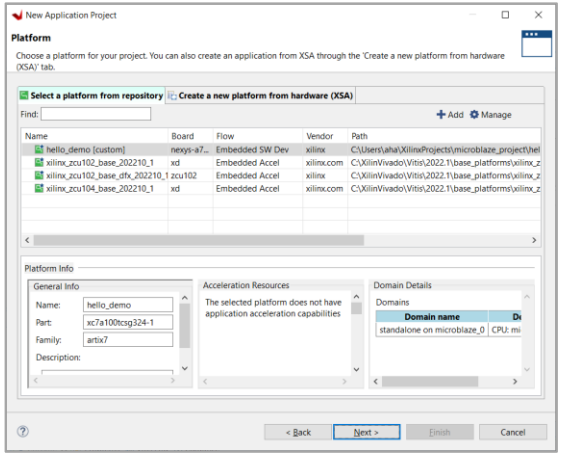


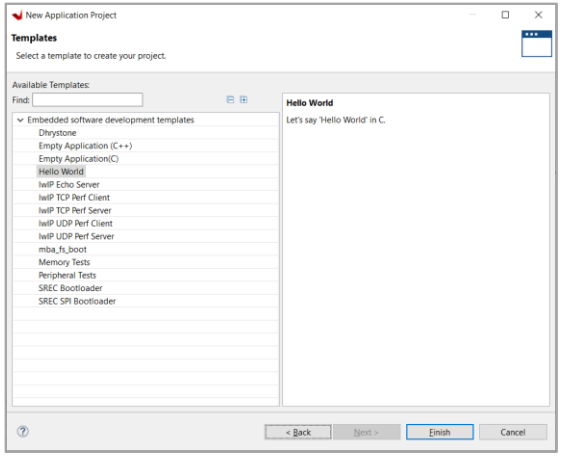
Hint

Building the platform compiles the drivers and the BSP for your hardware. The application can only be created and built after this step.

2.4 Create the application project

1. Select File, New, Application Project
2. Select the platform you have just built
3. Give the application a name and keep the default domain (standalone on microblaze_0)
4. Select the Hello World template
5. Click Finish





Hint

In this step the embedded application is created: Vitis generates the source files from a template, so you start with a program that already compiles and prints a message.

2.5 Edit the code and build the application

```
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"

int main()
{
    init_platform();
    print("Hello ELE Students\n\r");
    print("Successfully ran Hello World application");
    cleanup_platform();
    return 0;
}
```

Steps:

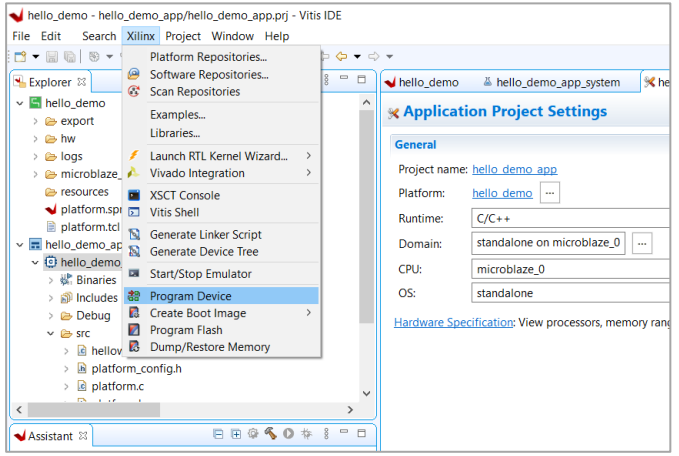
- Open src/helloworld.c in the Explorer
- Change the messages to your own text
- Right-click the application, select Build Project
- Check that the console reports no errors

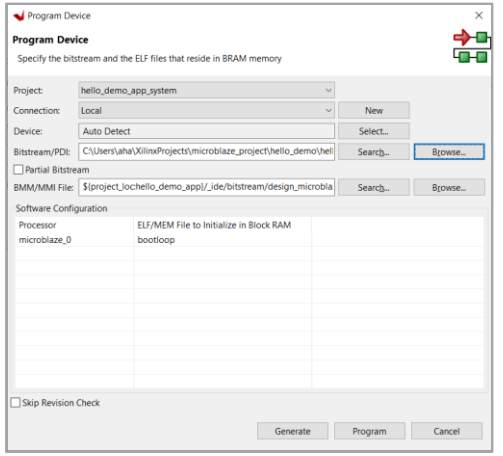
Hint

Compiling turns the C code into an ELF file, the executable the MicroBlaze runs. print() sends text to the UART; init_platform() sets up the board, here the UART.

2.6 Program the FPGA

1. Connect the board with the micro USB cable and switch it on
2. In Vitis select Xilinx, Program Device
3. Select the bitstream file (from the XSA export)
4. Click Program and wait until it is done





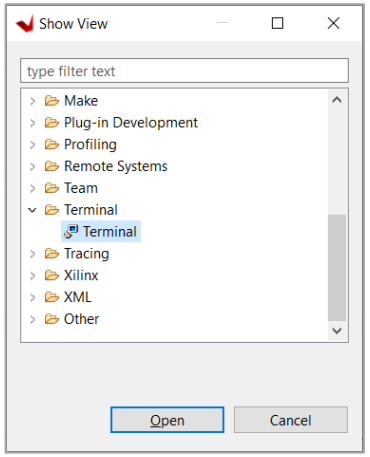
Hint

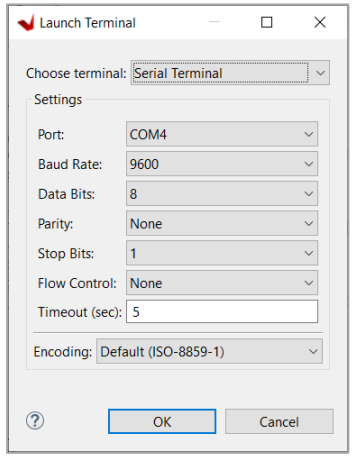
Programming loads the bitstream into the FPGA, which then becomes your MicroBlaze system. The configuration is lost when the board is powered off.

In newer versions you can just click Build and then Run

2.7 Open and configure the serial terminal

1. Open the terminal view (Window, Show View, Terminal)
2. Choose Serial Terminal and click Launch Terminal
3. Select the COM port of the board
4. Set the baud rate of your UART IP, 8 data bits, no parity, 1 stop bit



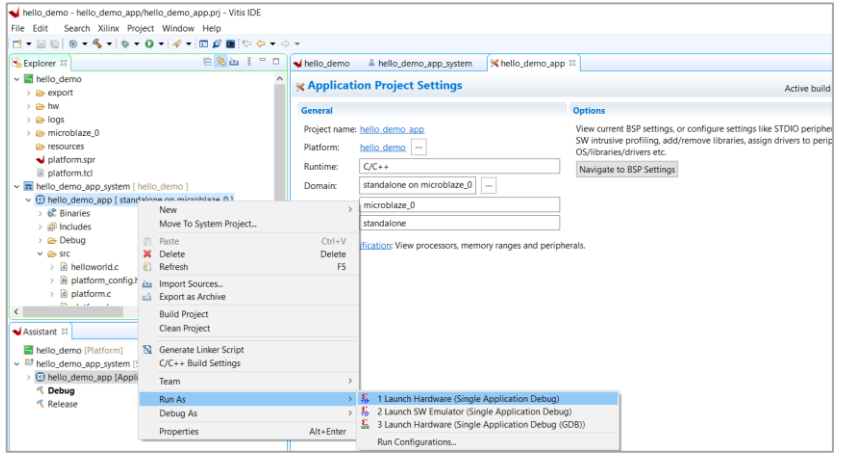


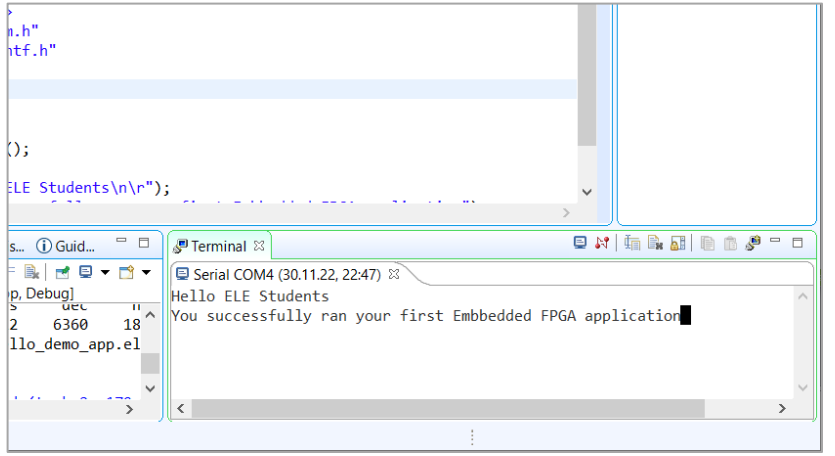
Hint

The terminal shows what your program sends over the UART. The baud rate must be the same on both sides: in the UART IP and in the terminal.

2.8 Run the application on the MicroBlaze

1. Right-click the application project
2. Select Run As, Launch on Hardware
3. Watch the terminal: your message appears´c





Hint

Run downloads the ELF into the local memory of the MicroBlaze through the debug module and starts the processor.

Task 3: echo program

```c
#include "xil_printf.h"

int main(void)
{
    xil_printf("Echo test: type a character\r\n");

    while (1) {
        char c = inbyte();
        xil_printf("You typed: %c\r\n", c);
    }

    return 0;
}
```

Task:

- Create a second application with the Hello World template
- Replace the code with the echo program
- Build, run it and type characters into the terminal
- Extend: capital letters, count characters

Hint

Now the UART is used in both directions: the program reads keystrokes from the PC and answers. This is the "reads the input from UART" part of the lab introduction.

Next steps…

1. Add an AXI GPIO for the LEDs and switches; mirror switches to LEDs.
2. Add an AXI Timer and blink an LED using the interrupt controller.
3. Add your own IP from Lab 1 and control it from C.

Lab 03!

Deliverables and summary

Item	Content
Block design	Screenshot of the IP Integrator diagram
Reports	Utilization and timing summary of the implementation
Software	Your C source and a screenshot of the terminal output
Demo	Show the message on the terminal
Summary	Hardware: block design, wrapper, bitstream, XSA. Software: platform, application, program, run.

Thank you for your attention!

Prof. Dr. -Ing. Ali Hayek

University of Applied Sciences Hamm-Lippstadt (HSHL)

Tel.: **+49 (2381) 8789 – 924**

E-Mail.: **ali.hayek@hshl.de**